

EXACT PROPAGATORS FOR THE GLIF₃ STEP

REPLACING THE EULER STEP ON THE SYNAPTIC CURRENT
IN THE V1 MODEL

The V1 cortical model advances each GLIF₃ neuron with a step of $\delta t = 1$ ms. The synaptic and after-spike currents obey linear equations, and the original code already used their exact one-step solutions; only the membrane update was approximate. It used a *Euler* step, which freezes the total synaptic current at its start-of-step value for the whole millisecond, and applies the reset and the after-spike current of a spike one step late. With a fastest synaptic basis of $\tau_1 = 0.99$ ms, that current rises and falls by an order of magnitude *inside* one step. Together the two approximations are the dominant error of the simulation — not, as one might assume, the `float16` arithmetic used during training — and at ~ 10 Hz the event timing contributes the larger share. We derive the exact one-step propagator for the coupled (synaptic, after-spike, membrane) system, following the construction of Rotter and Diesmann [1], and fold the spike events into the same constants. Against a `float64` ground truth obtained by a third, independent method, the relative RMS error of the membrane potential falls by a factor of 560 742, and the fraction of spikes reproduced at exactly the right millisecond rises from 7.1 % to 99.9 %. The new step requires no additional state variables and costs $1.09\times$ the time of the fused CUDA kernel it replaces. In the `float16` configuration training actually uses, the gain is $18\times$ rather than 560 742, and the binding constraint is no longer the integrator but the `float16` storage of the propagator constants — which, being shared across the batch and the sequence, would cost 12 MB in total to widen.

CONTENTS

1	The model and its discretisation	2
1.1	Continuous-time dynamics	2
1.2	The state vector	3
2	The previous scheme, and why it fails here	3
2.1	What was implemented	3
2.2	When an arriving spike takes effect	4
2.3	Why holding the current is not a small approximation	4

2.4	Two further inconsistencies in the event timing	5
2.5	Event conventions against the fitted model	5
3	The exact step	6
3.1	Propagator	6
3.2	Spike events	7
3.3	Removable singularities	7
3.4	The old scheme as a special case	8
3.5	Gradients	8
3.6	Verification of the closed forms	9
4	Accuracy	9
4.1	An independent ground truth	9
4.2	Error in the two regimes, at both precisions	10
4.3	A single postsynaptic potential	11
4.4	Spike-level fidelity	12
5	Cost	13
5.1	Arithmetic and memory	13
5.2	Measured kernel time	13
6	Consequences and open questions	14
6.1	Where the accuracy floor now lies	14
6.2	Two defects the rewrite exposed	15
6.3	Reproducing earlier work	16
6.4	What remains on the grid	16
6.5	Summary	16
7	References	17

1 THE MODEL AND ITS DISCRETISATION

1.1 Continuous-time dynamics

We follow the notation of [2]. A GLIF3 neuron [3, 4] carries a membrane potential $V(t)$ and two after-spike currents (ASCs) $I^s(t)$, $s \in \{1, 2\}$, obeying

$$\tau_m \frac{dV(t)}{dt} = -(V(t) - E_L) + R_m I(t), \quad \frac{dI^s(t)}{dt} = -k^s I^s(t), \quad (1)$$

with $\tau_m = R_m C_m$ and $I(t) = I^{\text{int}}(t) + I^{\text{ext}}(t)$ the total input current, where $I^{\text{int}}(t) = \sum_s I^s(t)$ (cf. [2], Eq. 1.8). At a spike time t , where $V(t) > V_{th}$,

$$V(t^+) \leftarrow E_L, \quad I^s(t^+) \leftarrow I^s(t) + \delta I^s, \quad (2)$$

with δI^s the constant ASC increment (cf. [2], Eq. 1.9).

The external current is generated by current-based synapses with alpha-function kernels. Writing $C^{\text{syn}} \equiv I^{\text{syn}} + I^{\text{syn}}/\tau_s$ reduces the second-order kernel to the first-order pair [1, 5]

$$\frac{dI^{\text{syn}}}{dt} = -\frac{I^{\text{syn}}}{\tau_s} + C^{\text{syn}}, \quad \frac{dC^{\text{syn}}}{dt} = -\frac{C^{\text{syn}}}{\tau_s} + \frac{e}{\tau_s} W_{ji} z_i(t - d_{ji}), \quad (3)$$

(cf. [2], Eq. 1.25), with z_i the presynaptic spike train and d_{ji} the axonal delay.

In the V1 model [6] each synapse is decomposed over $R = 4$ receptor bases with fixed time constants τ_r , so every neuron j carries a pair $(C_{j,r}^{\text{syn}}, I_{j,r}^{\text{syn}})$ per basis. The total input current is

$$I_j(t) = \sum_{r=1}^R I_{j,r}^{\text{syn}}(t) + \sum_{s=1}^2 I_j^s(t). \quad (4)$$

Normalising voltages by $\Delta V = V_{th} - E_L$ so that $E_L = 0$, $V_{th} = 1$ — as the implementation does — gives $R_m = 1/g$, with g the leak conductance, and Eq. (1) becomes $\dot{V} = -V/\tau_m + I/C_m$.

1.2 The state vector

Collecting the $2R + 3$ variables of one neuron into

$$\mathbf{y}_j = (C_{j,1}^{\text{syn}}, \dots, C_{j,R}^{\text{syn}}, I_{j,1}^{\text{syn}}, \dots, I_{j,R}^{\text{syn}}, I_j^1, I_j^2, V_j)^\top, \quad (5)$$

Eqs. (1)–(3) read $\dot{\mathbf{y}}_j = \mathbf{M}_j \mathbf{y}_j$ between spikes, with a constant, lower-triangular generator $\mathbf{M}_j$. This is the observation on which everything below rests: *between spikes the GLIF3 neuron is a linear time-invariant system*, so its one-step transition has a closed form and need not be approximated at all.

2 THE PREVIOUS SCHEME, AND WHY IT FAILS HERE

2.1 What was implemented

The implementation advanced the state over $[t, t + \delta t]$ as

$$C_{j,r}^{\text{syn}}[t + \delta t] = \alpha_r C_{j,r}^{\text{syn}}[t] + \frac{e}{\tau_r} \sum_{\text{src}} \sum_i W_{ji}^{\text{src}} z_i^{\text{src}}[t - d_{ji}], \quad (6)$$

$$I_{j,r}^{\text{syn}}[t + \delta t] = \alpha_r I_{j,r}^{\text{syn}}[t] + \delta t \alpha_r C_{j,r}^{\text{syn}}[t], \quad (7)$$

$$I_j^s[t + \delta t] = \beta_s I_j^s[t] + S_j[t] \delta I^s, \quad (8)$$

$$V_j[t + \delta t] = \alpha V_j[t] + (1 - \alpha) R_m I_j[t] - S_j[t] \Delta V, \quad (9)$$

with the decay factors $\alpha = e^{-\delta t/\tau_m}$, $\alpha_r = e^{-\delta t/\tau_r}$ and $\beta_s = e^{-k^s \delta t}$, and $S_j[t]$ the spike indicator of [2], Eq. 1.4. Equations (6)–(9) are Eqs. 1.27 and 1.10 of the thesis, specialised to $E_L = 0$.

Two points deserve emphasis before we criticise anything.

First, Eqs. (6)–(8) are *already exact*. For C^{syn} and I^s this is plain exponential decay. For I^{syn} it is less obvious: the term $\delta t \alpha_r C_{j,r}^{\text{syn}}[t]$ in Eq. (7) looks like C^{syn} held constant over the step, even though C^{syn} decays during it. Over one step with no arriving spike, Eq. (3) gives $C^{\text{syn}}(t + u) = e^{-u/\tau_r} C^{\text{syn}}(t)$ for $u \in [0, \delta t]$. Substituting

$$I^{\text{syn}}(t + u) = e^{-u/\tau_r} (I^{\text{syn}}(t) + u C^{\text{syn}}(t)) \quad (10)$$

into Eq. (3) gives $\frac{d}{du} I^{\text{syn}} = -I^{\text{syn}}/\tau_r + e^{-u/\tau_r} C^{\text{syn}}(t) = -I^{\text{syn}}/\tau_r + C^{\text{syn}}(t + u)$. Eq. (10) therefore solves the equation exactly, time-varying C^{syn} included. At $u = \delta t$ it is Eq. (7). It is exact because C^{syn} and I^{syn} share the time constant τ_r . Once that common decay is factored out, $e^{u/\tau_r} C^{\text{syn}}(t + u) = C^{\text{syn}}(t)$ really is constant over the step, and the factor δt is its exact integral. As an example, take $\tau_r = \delta t = 1$ ms,

$I^{\text{syn}}(t) = 0$ and $C^{\text{syn}}(t) = 1$. Then Eq. (7) gives $I^{\text{syn}}(t + \delta t) = e^{-1} \approx 0.37$, the exact value. A genuine forward-Euler step, $I^{\text{syn}} + \delta t(C^{\text{syn}} - I^{\text{syn}}/\tau_r)$, would give 1. Arriving spikes enter C^{syn} at a step boundary, which is exact for events on the time grid (Section 2.2). After every step, all synaptic and after-spike states hold their exact values, and nothing in their update needs improving.

Second, Eq. (9) is not forward Euler either: the homogeneous part carries the exact factor $e^{-\delta t/\tau_m}$ rather than $1 - \delta t/\tau_m$. It is an *exponential Euler* step, and it would be exact if I_j were constant across the step. The approximation is entirely in that last clause. We refer to Eq. (9) throughout as the *Euler* scheme, following the thesis, and note that what it approximates is a *zero-order hold* on the current: I_j is held at its start-of-step value and the factor $1 - \alpha = \int_0^{\delta t} e^{-(\delta t - u)/\tau_m} du / \tau_m$ is the integral of the membrane kernel against that constant.

2.2 When an arriving spike takes effect

In Eq. (6) the new input is added to C^{syn} without being decayed, and Eq. (7) uses the old $C_{j,r}^{\text{syn}}[t]$. Both are exact if the spike arrives at the end of the step, $t + \delta t$. The delta in Eq. (3) makes C^{syn} jump by eW_{ji}/τ_r at the arrival time, while I^{syn} , the integral of C^{syn} , is continuous. A jump at $t + \delta t$ has no time left to decay within the step and has not yet contributed to I^{syn} . From the next step on, it decays and feeds I^{syn} through Eq. (10).

Placing the arrival at the end of the step is what makes the delays come out right. Let a presynaptic neuron i spike at grid point t_n . That spike is computed from $V_i[t_n]$, and the synapse onto j , with delay $d_{ji} = d\delta t$, reads it $d - 1$ steps later. The jump then lands at the end of that step, at $t_n + d\delta t$. This is exactly the delay in the data. Had the jump been placed at the start of the same step, every synapse would act one step early, and the shortest delay of 1 ms would become instantaneous. Both conventions solve Eq. (3) exactly; they differ only in the arrival time they assign. The afferent LGN input follows the same rule.

The neuron's own spike is the opposite case. It happens at t_n , the start of the step that follows it, so its reset and after-spike increment belong at the start of that step (Section 2.4):

event	occurs at	enters the step
own spike: reset and δI^s	t_n	at the start of $[t_n, t_{n+1})$
arrival at j with delay $d\delta t$	$t_n + d\delta t$	at the end of $[t_{n+d-1}, t_{n+d})$

2.3 Why holding the current is not a small approximation

Holding the current constant is defensible when it varies slowly compared with δt . Figure 1 shows that in this model it does not. The four receptor bases have $\tau_r \in \{0.99, 1.88, 3.63, 5.65\}$ ms against $\delta t = 1$ ms, while the membrane constants span 5.5–51.8 ms (median 14.3 ms) across the 201 cell types. The alpha kernel driven by the fastest basis peaks at $t = \tau_1 \approx \delta t$: it rises from zero to its maximum and begins to decay *within the first step after the presynaptic spike*.

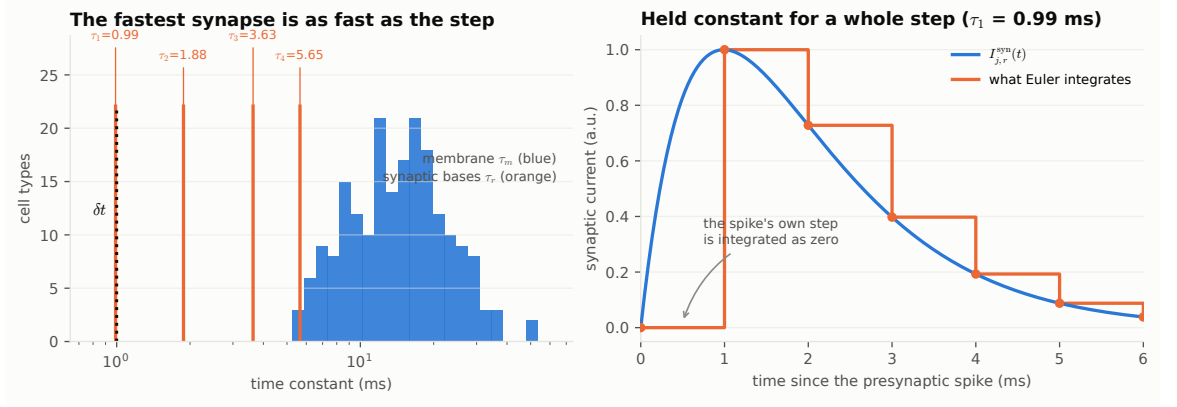


FIGURE 1: **The synaptic timescales collide with the step.** *Left:* distribution of membrane time constants τ_m over the 201 GLIF3 cell types (blue), with the four receptor-basis constants τ_r (orange) and the step δt (dotted). *Right:* the alpha current $I_{j,1}^{\text{syn}}(t)$ produced by one presynaptic spike on the fastest basis (blue), against the staircase the Euler step integrates in its place (orange). Because $I_{j,r}^{\text{syn}}(t_s) = 0$ at the arrival time, Euler assigns the spike's own step a current of exactly zero.

The right panel of Figure 1 makes the failure concrete. A newly arrived spike sets $C_{j,r}^{\text{syn}}$, not $I_{j,r}^{\text{syn}}$; the latter is still zero at the grid point. Euler therefore integrates the entire first millisecond of every postsynaptic current as *zero*, and then lags by one step thereafter.

2.4 Two further inconsistencies in the event timing

The same audit exposes two off-by-one-step errors in how spike events enter Eqs. (8)–(9). A postsynaptic spike is emitted at grid point t from $V_j[t]$, so its consequences act *at* t and must then propagate across $[t, t + \delta t)$:

- the reset in Eq. (9) subtracts ΔV at the *end* of the step, giving $\alpha V_j[t] - \Delta V$. A reset at t gives $\alpha(V_j[t] - \Delta V)$ instead, a discrepancy of $(1 - \alpha)\Delta V \approx 0.07 \Delta V$ at the median τ_m ;
- the ASC increment in Eq. (8) is likewise added after the decay, and the freshly injected ASC does not drive V_j during the step in which it appears.

2.5 Event conventions against the fitted model

The GLIF3 parameters were fitted by Teeter *et al.* [4], and the fitting procedure fixes where each spike event belongs. The fitting code¹ regresses the sub-threshold voltage on basis currents $e^{-k^s(t-t_{\text{sp}})}$ that start at spike initiation t_{sp} , and only removes the samples inside the refractory window (the “spike cut”) from the regression. The fitted ASC is therefore $\delta I^s e^{-k^s(t-t_{\text{sp}})}$ from the spike onwards, and a current present before the spike decays through the window, as in their supplementary Eq. 6. The membrane is not modelled inside the window and restarts at $V_{\text{reset}} = E_L$ when it ends.

¹ <https://github.com/AllenInstitute/AllenSDK/blob/master/allensdk/internal/model/glif/ASGLM.py> and `create_basis_IPSP` in <https://github.com/AllenInstitute/AllenSDK/blob/master/allensdk/internal/model/GLM.py>.

The exact step follows this on the grid. A spike detected at t resets the membrane and injects δI^s at the start of $[t, t + \delta t)$, and both then evolve through the step (Eq. (16)). Further spikes are blocked until $t + \lceil t_{\text{ref}}/\delta t \rceil \delta t$. Because the true crossing lies in $(t - \delta t, t]$, that is the first grid point that can follow the end of the refractory window. Under a hard reset the membrane is held at V_{reset} up to the grid point before it. A trace of one spike through both backends, for both resets, reproduces these values to within 10^{-6} of the corresponding matrix-exponential solution.

The implementation departs from Eq. (2) in one deliberate respect. For training it resets by subtraction, $V \leftarrow V - \Delta V$, rather than to E_L , and lets the membrane integrate its input during the refractory period while only spiking is blocked. This keeps the overshoot $V_j[t] - V_{th}$ and the information the input carries through the refractory period. It is also a departure from the fit: the after-spike currents keep driving the membrane during the refractory period, while the fitted model does not simulate the membrane there and the hard reset holds it. Only with no input, no overshoot and no after-spike current does the soft reset return to E_L as the hard reset does. The hard reset, which reproduces the fitted model on the δt grid, remains available. It used to fall back to the subtractive rule when the refractory period was a single step; it now resets to V_{reset} in that case too. The current network is not affected, since its shortest t_{ref} is 1.05 ms.

NEST's `glif_psc` [7]² also resets at the grid point where the crossing is detected, so the reset timing agrees. It differs in two ways, both inherited from the Allen simulator. It adds δI^s undecayed at the *end* of the refractory window, which does not match the fitted current above. It also counts the refractory period from the detection time as if the crossing happened exactly there, which delays the next spike by one step whenever the fractional part of t_{ref} is zero or at least one half.

3 THE EXACT STEP

3.1 Propagator

Since $\dot{\mathbf{y}}_j = \mathbf{M}_j \mathbf{y}_j$ is linear and time invariant, the exact transition over one step is the matrix exponential $\mathbf{y}_j[t + \delta t] = e^{\mathbf{M}_j \delta t} \mathbf{y}_j[t]$. This is the propagator-matrix construction of Rotter and Diesmann [1], used by NEST for `iaf_psc_alpha` [7]. Only the last row is new: the first $2R + 2$ rows reproduce Eqs. (6)–(8).

Integrating the membrane equation exactly rather than under a hold gives a convolution in place of a product,

$$V_j(t + \delta t) = e^{-\delta t/\tau_m} V_j(t) + \frac{1}{C_m} \int_0^{\delta t} e^{-(\delta t - u)/\tau_m} I_j(t + u) du, \quad (11)$$

² https://github.com/nest/nest-simulator/blob/master/models/glif_psc.cpp,
function `glif_psc::update`.

and because each source in Eq. (4) is a known exponential or alpha function on $[t, t + \delta t)$, every integral is a constant. Writing $a = 1/\tau_m$ and $b_r = 1/\tau_r$, the resulting step is

$$\boxed{V_j[t + \delta t] = \alpha V_j[t] + \sum_{r=1}^R \left(\mathcal{A}_{j,r} I_{j,r}^{\text{syn}}[t] + \mathcal{B}_{j,r} C_{j,r}^{\text{syn}}[t] \right) + \sum_{s=1}^2 \mathcal{D}_j^s I_j^s[t] + (\mathcal{R}_j + \mathcal{S}_j) S_j[t]} \quad (12)$$

with the coefficients

$$\mathcal{A}_{j,r} = \frac{1}{C_m} \int_0^{\delta t} e^{-a(\delta t - u)} e^{-b_r u} du = \frac{1}{C_m} \frac{e^{-b_r \delta t} - e^{-a \delta t}}{a - b_r}, \quad (13)$$

$$\mathcal{B}_{j,r} = \frac{1}{C_m} \int_0^{\delta t} e^{-a(\delta t - u)} u e^{-b_r u} du = \frac{1}{C_m} \left[\frac{\delta t e^{-b_r \delta t}}{a - b_r} - \frac{e^{-b_r \delta t} - e^{-a \delta t}}{(a - b_r)^2} \right], \quad (14)$$

$$\mathcal{D}_j^s = \frac{1}{C_m} \int_0^{\delta t} e^{-a(\delta t - u)} e^{-k^s u} du = \frac{1}{C_m} \frac{e^{-k^s \delta t} - e^{-a \delta t}}{a - k^s}. \quad (15)$$

3.2 Spike events

A postsynaptic spike does two distinct things to its own step, and applying both at the spike's grid point, then propagating them across the step, gives, for the subtractive reset the implementation uses (Section 2.5),

$$\underbrace{\mathcal{R}_j = -\alpha \Delta V}_{\text{the reset}}, \quad \underbrace{\mathcal{S}_j = \sum_{s=1}^2 \mathcal{D}_j^s \delta I^s}_{\text{the ASC it injects}}, \quad I_j^s[t + \delta t] = \beta_s I_j^s[t] + \underbrace{\beta_s \delta I^s}_{\widehat{\delta I^s}} S_j[t]. \quad (16)$$

$\mathcal{R}_j$ is the reset, which now decays across the step rather than being subtracted at its end. $\mathcal{S}_j$ is the drive the freshly injected after-spike current contributes to the membrane during that same step — a term the Euler scheme did not have at all, because there the ASC only reached the membrane on the following step.

Both multiply the same $S_j[t]$, so the forward pass could fold them into one constant and save an operation. They are deliberately kept apart, because the two training flags gate them separately. The flag `detach_reset` cuts the gradient path from the spike to the reset, and `detach_asc_reset` cuts the path from the spike to the ASC. Folding them would make `detach_reset` silently cut part of the ASC gradient too — and $\mathcal{S}_j$ reaches 28 % of $|\mathcal{R}_j|$ for some cell types, so that is not a rounding matter. Apart from that one extra multiply-add, Eq. (12) has the same operation count on the spike term as Eq. (9).

3.3 Removable singularities

Both $\mathcal{A}_{j,r}$ and $\mathcal{D}_j^s$ carry a removable singularity at $a = b_r$ (respectively $a = k^s$), and $\mathcal{B}_{j,r}$ a double one. This is not a hypothetical concern: the slowest basis, 5.65 ms, lies inside the membrane range 5.5–51.8 ms, so

some cell types land on the degeneracy to within rounding. Writing $x = (a - b_r)\delta t$ and

$$\mathcal{A}_{j,r} = \frac{\delta t e^{-a\delta t}}{C_m} \cdot \frac{e^x - 1}{x}, \quad \mathcal{B}_{j,r} = \frac{\delta t^2 e^{-a\delta t}}{C_m} \cdot \frac{e^x(x-1) + 1}{x^2}, \quad (17)$$

both factors are entire, and at $x = 0$ reduce to $\mathcal{A} = \delta t e^{-a\delta t} / C_m$ and $\mathcal{B} = \frac{1}{2} \delta t^2 e^{-a\delta t} / C_m$.

Being entire is not the same as being computable, and the second factor is a trap. Written as it stands, $e^x(x-1) + 1$ subtracts two quantities of size 1 to produce one of size $x^2/2$, so it forfeits $2 \log_{10}(1/|x|)$ significant digits — eight of them at $x = 10^{-4}$. The band is reached in practice: across the 201 cell types the smallest $|x|$ is 1.8×10^{-3} . Rewriting through

$$\frac{e^x(x-1) + 1}{x^2} = 1 + (x-1) \varphi_2(x), \quad \varphi_2(x) = \frac{1}{x} \left(\frac{e^x - 1}{x} - 1 \right), \quad (18)$$

leaves a cancellation of 1 against $1/2$ rather than against $x^2/2$, which costs one order of x instead of two. Below a crossover the Taylor series $\sum_n x^n(n+1)/(n+2)! = \frac{1}{2} + \frac{x}{3} + \frac{x^2}{8} + \frac{x^3}{30} + \frac{x^4}{144} + \dots$ takes over; the crossover is placed where the series truncation meets the recurrence's rounding. The worst relative error over all x falls from 1.8×10^{-8} to 3.6×10^{-14} , and $\mathcal{B}$ then matches $e^{M_j \delta t}$ to 10^{-16} for every cell type. The first factor, $\frac{e^x - 1}{x} = 1 + \frac{x}{2} + \frac{x^2}{6} + \dots$, needs no such care: `expm1` evaluates it to machine precision directly.

3.4 The old scheme as a special case

Setting

$$\begin{aligned} \mathcal{A}_{j,r} &= \mathcal{D}_j^s = (1 - \alpha) R_m, & \mathcal{B}_{j,r} &= 0, \\ \mathcal{R}_j &= -\Delta V, & \mathcal{S}_j &= 0, & \widetilde{\delta I^s} &= \delta I^s \end{aligned} \quad (19)$$

turns Eq. (12) back into Eq. (9) exactly. The integration scheme is therefore a *choice of constants*, not a branch in the code: both the TensorFlow path and the fused CUDA kernel evaluate Eq. (12) unconditionally, and the historical behaviour is recovered bit for bit by building the coefficients of Eq. (19) at setup time.

3.5 Gradients

The model is trained by backpropagation through time, so the propagator is only as useful as its Jacobian. Differentiating Eq. (12) and the state updates gives, for upstream gradients $\bar{V}, \bar{I}^s, \bar{C}, \bar{I}$ on the four outputs,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial V_j[t]} &= \alpha \bar{V}, & \frac{\partial \mathcal{L}}{\partial I_{j,r}^{\text{syn}}[t]} &= \alpha_r \bar{I}_r + \mathcal{A}_{j,r} \bar{V}, \\ \frac{\partial \mathcal{L}}{\partial I_j^s[t]} &= \beta_s \bar{I}^s + \mathcal{D}_j^s \bar{V}, & \frac{\partial \mathcal{L}}{\partial C_{j,r}^{\text{syn}}[t]} &= \alpha_r \bar{C}_r + \delta t \alpha_r \bar{I}_r + \mathcal{B}_{j,r} \bar{V}, \end{aligned} \quad (20)$$

and for the incoming spike, with the two detach flags written as indicators,

$$\frac{\partial \mathcal{L}}{\partial S_j[t]} = \mathbb{1}_{\text{attach reset}} \mathcal{R}_j \bar{V} + \mathbb{1}_{\text{attach ASC}} \left(\mathcal{S}_j \bar{V} + \sum_s \widetilde{\delta I^s} \bar{I^s} \right). \quad (21)$$

Every new coefficient appears in the backward pass exactly where it appears in the forward one, so the added cost is the same handful of multiply-adds. Equation (21) is where the separation of $\mathcal{R}_j$ and $\mathcal{S}_j$ earns its keep: each flag gates its own term.

The TensorFlow path is built from plain operations and is differentiated by autodiff, but the CUDA path carries a hand-written backward kernel, so both were checked against central finite differences of the forward step (`verify_gradients.py`). Two inputs are excluded from that check because a finite difference across them measures a step function rather than the transition: the emitted spikes, which pass through a Heaviside with a surrogate gradient, and the delayed spike buffer, which feeds the event-driven recurrent kernel where a spike is consumed as a binary event. Both are pre-existing choices of the model, independent of the integrator.

Over the remaining, smooth inputs — V_j , $C_{j,r}^{\text{syn}}$, $I_{j,r}^{\text{syn}}$, I_j^s and the incoming spike — both backends and both schemes agree with finite differences to a worst relative discrepancy of 1.9×10^{-2} , which is the float32 difference-quotient noise floor at this step size; the spike gradient of Eq. (21), which is exactly linear and therefore free of truncation error, agrees to 8.1×10^{-5} . The CUDA and TensorFlow gradients agree with each other to float32 rounding, which the test suite asserts independently.

3.6 Verification of the closed forms

Equations (13)–(15) were checked against $e^{M_j \delta t}$ computed by scaling-and-squaring [8] (`scipy.linalg.expm`) for every fifth cell type. The largest discrepancy in any coefficient is below 10^{-16} — machine precision, and an independent confirmation of the algebra, since the two routes share no code. (Its last digits depend on the LAPACK build, so only the order is quoted.)

4 ACCURACY

4.1 An independent ground truth

Measuring the exact scheme against a reference built *from* the exact propagator would be circular. We therefore construct the ground truth from the *old* method: the Euler step is first-order accurate, so sub-stepping it at $\delta t/M$ converges to the true trajectory as $\mathcal{O}(1/M)$, and a Richardson extrapolation [9]

$$V^{\text{ref}} = 2 V_{2M}^{\text{euler}} - V_M^{\text{euler}}, \quad M = 128, \quad (22)$$

cancels the leading error term, leaving a reference accurate to $\sim 10^{-7}$ that never touches the closed form under test. The reference takes only Euler’s *integration* from the old method: it applies the spike events at the start of each step, as the exact scheme does (Section 2.5), so it measures only the integration error. The event convention is part of the model’s definition, fixed by the fit, not an approximation

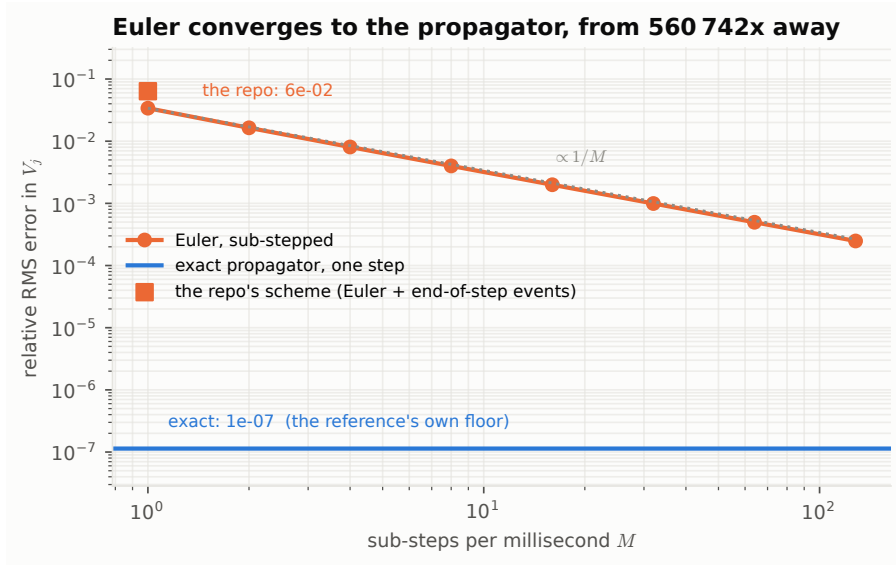


FIGURE 2: **Euler converges to the propagator.** Relative RMS error in V_j against the Richardson reference of Eq. (22), as the Euler step is sub-stepped at $\delta t/M$ (orange circles). The square marks the scheme as implemented, which additionally carries the end-of-step event timing of Section 2. The exact propagator (blue) sits at the reference’s own floor for $M = 1$.

TABLE 1: Relative RMS error in V_j against the Richardson reference. “state” narrows only the four recurrent state blocks; “+ constants” also narrows the propagator coefficients, which is what the kernel really does under a mixed-precision policy. The float64 entries for the exact scheme are bounded by the reference, not by the scheme, so the improvement factor in that column is a lower bound.

Regime	Scheme	float64	float16		Improvement
			state	+ constants	
Subthreshold	Euler	1.7×10^{-2}	1.7×10^{-2}	1.8×10^{-2}	$39\,871\times$
	exact propagator	4.2×10^{-7}	6.6×10^{-4}	3.6×10^{-3}	
Spiking (10.7 Hz)	Euler	6.4×10^{-2}	6.4×10^{-2}	6.4×10^{-2}	$560\,742\times$
	exact propagator	1.1×10^{-7}	1.0×10^{-3}	3.6×10^{-3}	

a convergent method could arbitrate. All errors below are relative RMS errors in V_j , in units of ΔV , over 201 cell types driven by a fixed Poisson input.

Figure 2 shows the sub-stepped Euler step walking towards the propagator as $1/M$ and never overtaking it. The propagator is already at the reference’s own accuracy floor at $M = 1$.

4.2 Error in the two regimes, at both precisions

Table 1 and Figure 3 separate two effects. The *subthreshold* regime has no postsynaptic spikes, so the error is purely the membrane integration of the synaptic current; the *spiking* regime adds the reset and the ASCs, whose event timing the exact scheme also corrects. Each is evaluated in float64, which isolates the method, and with the state stored in float16 and the arithmetic performed in float32 — exactly what the production CUDA kernel does [10].

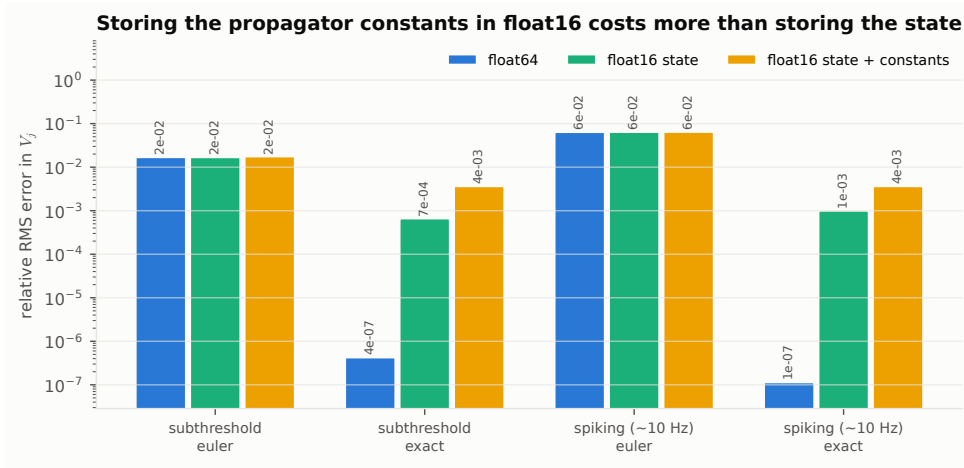


FIGURE 3: **The method error dominated the precision error.** Under Euler, moving from float64 to float16 storage changes nothing: the discretisation error is some forty times larger than anything the number format contributes. Only once the integrator is exact does float16 storage become the accuracy floor.

The float64 columns are the substantive result. Euler’s error, 6.4×10^{-2} of ΔV , is over 6 % of the distance from rest to threshold — a systematically wrong trajectory, not a rounding artefact. The three columns read together settle the question of precision, and reverse the expected answer twice. **Mixed precision was never the integrator’s bottleneck — under Euler, narrowing the state changes nothing, because the discretisation error is some forty times larger.** But once the integrator is exact, precision becomes the floor, and it is not the state that dominates it: narrowing the propagator *constants* costs a further $3.6\times$ on top of narrowing the state. Section 6.1 returns to this, because the constants are by far the cheapest thing here to widen.

Euler’s float64 error has two sources: the current held over the step (Section 2), and the spike events applied at the end of the step (Section 2.4). They live in disjoint constants, the current in $\mathcal{A}, \mathcal{B}, \mathcal{D}$ and the events in $\mathcal{R}, \mathcal{S}$ and the ASC increment, so each can be fixed on its own:

remaining approximation	subthreshold	spiking
both (Euler)	1.7×10^{-2}	6.4×10^{-2}
event timing only	4.2×10^{-7}	4.9×10^{-2}
current hold only	1.7×10^{-2}	2.6×10^{-2}
neither (exact)	4.2×10^{-7}	1.1×10^{-7}

Without spikes the current hold is the entire error. At the imposed ~ 10 Hz output rate, the event timing is the larger of the two, and neither fix alone gets within two orders of magnitude of the exact step. The split depends on the rate: the event error scales with the number of spikes, the hold error with the synaptic drive.

4.3 A single postsynaptic potential

Figure 4 isolates the mechanism on one neuron ($\tau_m = 16.2$ ms) receiving a single presynaptic spike. Euler cannot see the current rise inside the step in which it happens, so the postsynaptic potential starts

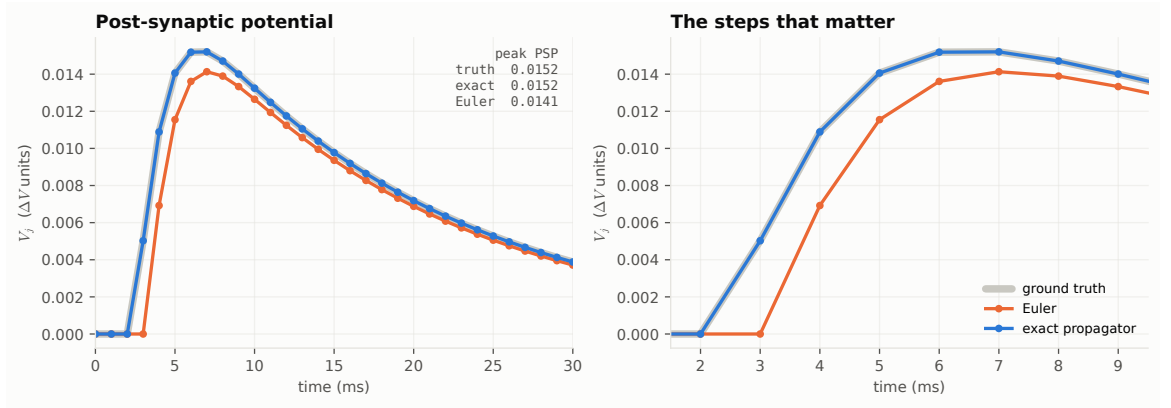


FIGURE 4: **One presynaptic spike, one neuron.** Membrane response to a single spike on the fastest receptor basis at $t = 2$ ms. The grey trace is the sub-stepped float64 ground truth; the exact propagator (blue) lies on top of it. *Right:* the first steps, where Euler (orange) integrates the arrival step as zero.

a millisecond late and underestimates the peak by 7.0 %. The exact propagator is indistinguishable from the ground truth.

4.4 Spike-level fidelity

Voltage error matters only insofar as it moves spikes. We therefore close the loop: neurons integrate, cross threshold, reset, adapt and block during their refractory period, driven by a fixed Poisson input at 10.7 Hz (1 723 reference spikes over 800 ms). The metric is deliberately strict — the fraction of ground-truth spikes reproduced in *exactly* the same millisecond — because a closed loop amplifies any voltage discrepancy into a different spike time, after which the trajectories separate.

The result (Figure 5) is that Euler reproduces 7.1 % of the ground-truth spikes and the exact propagator 99.9 %. In the float16 configuration training uses — state *and* constants — the exact scheme reproduces 61.9 %. Richardson extrapolation cannot supply this ground truth: once neurons spike on their own, runs at M and $2M$ sub-steps can diverge by a whole spike, and a raster cannot be extrapolated. It is therefore plain sub-stepped Euler at $M = 4096$, whose voltage error, first order in $1/M$, is about 1.6×10^{-5} of ΔV . That is small next to Euler’s error but not next to the exact scheme’s, so part of the exact scheme’s float64 mismatch is the reference’s own: against a reference with $M = 512$, the same run agrees on only 99.7 % of spikes. The float64 agreement of the exact scheme is therefore a lower bound. The 61.9 % is a strict count, and most of the remaining spikes are not far off: of the 657 ground-truth spikes the float16 run misses, 59 % have a float16 spike exactly one millisecond away, so 84 % of all ground-truth spikes are reproduced to within ± 1 ms. That remaining gap is storage precision, not the integrator, and Section 6.1 shows most of it is recoverable for a fixed 12 MB.

It is worth stating what this does *not* claim. Euler is stable and its mean firing rate is only 3.5 % off, so a network trained under it is a perfectly self-consistent dynamical system. It is simply not the GLIF3 system its parameters describe. Any quantity that depends on spike

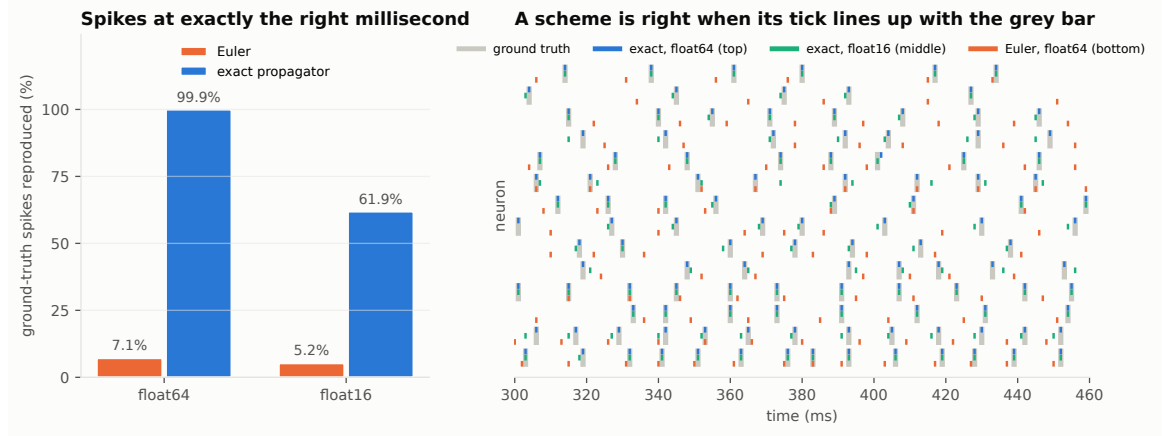


FIGURE 5: **Spikes land in the right millisecond.** *Left:* fraction of ground-truth spikes reproduced at exactly the right millisecond. *Right:* rasters for the fourteen most active neurons over a 160 ms window, with the exact propagator in float64 (top), the exact propagator in the production float16 configuration (middle) and Euler in float64 (bottom); a scheme is correct where its tick aligns with the grey ground-truth bar. The ground truth here is the *old* method sub-stepped at $\delta t/4096$ in float64, so it favours neither scheme. It applies the spike events at the start of each step, as the exact scheme does, so it measures only the integration error. Its own voltage error is about 1.6×10^{-5} of ΔV , so the exact scheme’s float64 agreement is a lower bound.

timing — synchrony, phase, latency, temporal codes, or the surrogate gradients that flow through the threshold crossing [11] — was being computed for the wrong model.

5 COST

5.1 Arithmetic and memory

Equation (12) replaces one scaled sum by a weighted sum: per neuron per step, R additional fused multiply-adds, and per-basis coefficient arrays where there was previously one scalar $(1 - \alpha)R_m$. The coefficients are shared across the batch and so remain resident in L2. Critically, $C_{j,r}^{\text{syn}}$ was *already* read by the kernel, which has to update it anyway — so the exact step introduces **no new state variables**, and therefore no additional memory on the backpropagation-through-time tape, which is what constrains this model in practice. The only new memory is 4.6 MB of constants at 230 924 neurons in float16, independent of batch size and sequence length.

5.2 Measured kernel time

Table 2 compares the fused forward and backward CUDA kernels before and after the change. Each timing chains 64 state transitions inside a single `tf.function`, so it measures per-timestep kernel cost rather than dispatch overhead.

In float16 the fused state kernel costs $1.09\times$ its previous time (range $1.06\text{--}1.11\times$). This is the cost of that one kernel, not of a training step: the state transition is one of several kernels per timestep and the recur-

TABLE 2: Per-timestep cost of the fused GLIF state kernels, in microseconds, on an NVIDIA RTX PRO 6000. Each timing chains 64 state transitions inside one `tf.function`. `fp16` is the configuration used for training.

Neurons	Batch	dtype	forward		backward		Ratio
			Euler	exact	Euler	exact	
51 978	8	fp16	15.3	16.6	20.0	20.9	$1.06\times$
51 978	8	fp32	26.4	33.1	32.7	36.7	$1.18\times$
51 978	32	fp16	50.5	55.7	66.2	71.2	$1.09\times$
51 978	32	fp32	116.9	135.0	141.7	150.8	$1.11\times$
230 924	8	fp16	57.4	64.1	73.8	81.2	$1.11\times$
230 924	8	fp32	136.8	152.2	156.2	165.6	$1.08\times$
230 924	32	fp16	293.4	315.4	345.5	379.5	$1.09\times$
230 924	32	fp32	580.2	647.1	658.9	690.8	$1.08\times$

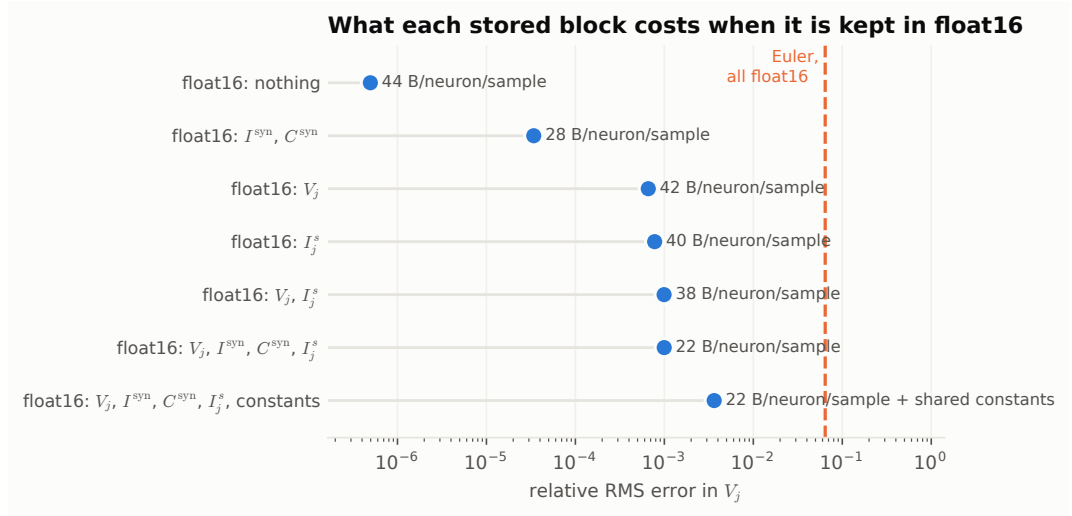


FIGURE 6: **What each stored block costs in float16.** Relative RMS error in V_j for the exact scheme, as each block is moved to float16. The four state blocks are charged per neuron per sample; the constants are shared across the batch and the sequence, so they add nothing to that column. The dashed line is Euler with everything narrowed.

rent CSR products dominate. An end-to-end training measurement has not been performed.

The overhead is not irreducible. The inner loop reads four separate $[N \times R]$ constant arrays at the same index; interleaving them into a single array of four values per (j, r) would turn four strided loads into one float4 fetch. That is a worthwhile optimisation, not a prerequisite.

6 CONSEQUENCES AND OPEN QUESTIONS

6.1 Where the accuracy floor now lies

With the method error removed, float16 storage sets the floor. Figure 6 attributes it, block by block, over the five things the kernel stores: the four recurrent state blocks and the propagator constants.

Two results here are worth separating, because they point the same way.

Among the *state* blocks the cost is counter-intuitive: I^{syn} and C^{syn} — eight of the eleven words per neuron — cost almost nothing in `float16`, while I^s , two words, costs more than both together. The after-spike currents carry large amplitudes and decay over hundreds of milliseconds, so their `float16` ulp is coarse and the error accumulates for a long time before it decays. Widening V_j and I_j^s to `float32` buys a 29-fold gain for 27 % more state on this block — a far smaller fraction of the full backpropagation-through-time tape, which is dominated by the spike ring buffer.

The *constants* are the better bargain, and were the easier thing to overlook. Narrowing them costs $3.6\times$ on its own, more than every state block put together, and it is the single largest term in the production number of Table 1. But unlike the state they do not scale: there are 25 words per neuron, shared across every sample in the batch and every step of the sequence, so widening all of them to `float32` at 230 924 neurons costs a fixed 12 MB. The obstacle is not memory but plumbing — the fused operator takes one dtype for every tensor it is given, so mixed state and constant widths need a second type parameter on the op.

We quantify both trades here and take neither: the integrator fix is unconditional, while the precision choices are deliberate decisions with their own costs, and the constants one needs a kernel signature change. On the evidence above the constants should be widened first.

6.2 Two defects the rewrite exposed

Reworking the setup code turned up two bugs that predate the propagator and have nothing to do with it. Both are fixed here.

THE ASC DECAY WENT THROUGH `float16`. $\beta_s = e^{-k^s \delta t}$ was not computed from k^s directly. The rate was cast to the compute dtype, pushed through an inverse sigmoid, stored, and pushed back through a sigmoid on read — a parameterisation left over from when k was to be trained. Under a mixed-precision policy that round trip happens in `float16` and returns k^s with a relative error of 2.0×10^{-3} . The error in β_s looks negligible, at 3.6×10^{-5} , but it is an error in a *decay rate*: over three ASC time constants it compounds to 0.6 % of the after-spike current’s amplitude, and the slowest ASC has $\tau = 333$ ms, so that is a sustained bias on the slowest adaptation in the model. It is now evaluated in `float64` and cast once.

A NEURON-ALIGNED CONSTANT WAS NOT FOLLOWING THE LAYOUT. The model can renumber neurons into a spatially local (Morton) order at runtime while keeping checkpoints in canonical order, and `_translate_neuron_layout` permutes the neuron-aligned arrays when it does. It worked from a hand-written list of attribute names, and `t_ref_steps` was not on it — so under a non-identity layout every neuron got somebody else’s refractory period, drawn from a range of 2 to 10 steps. The read is live in both backends.

The second is the more instructive failure: a list of names that has to be kept in step with a set of attributes will eventually fall out of step. The edge-aligned translation next to it does not have this problem,

because it discovers its targets. The per-neuron constants are now built by a helper that binds and registers in one call, so the registry cannot disagree with what exists, and a test asserts that every constant the step reads is in it and survives a permutation round trip.

6.3 Reproducing earlier work

The coefficients are derived constants, not trainable parameters, and are deliberately excluded from the checkpoint graph, so checkpoints written before this change still restore and are interchangeable between the two schemes. What is *not* interchangeable is the science: a network whose weights were trained under Euler was fitted to a different dynamical system, and its firing rates will shift when evaluated under the exact step. Earlier results should be reproduced with the Euler coefficients of Eq. (19), and new models retrained.

6.4 What remains on the grid

Spikes are still emitted and delivered only at millisecond boundaries. Morrison *et al.* [12] show how to carry continuous spike times through the same propagator formalism, and Hanuschkin *et al.* [13] give a general time-driven implementation; either would remove the remaining sub-millisecond quantisation. Its largest visible effect is on the after-spike currents. A spike is detected at the first grid point after the true crossing, but δI^s is injected there at full amplitude. The fitted current has already decayed from spike initiation, so for the fastest ASC ($k^s = 0.3 \text{ ms}^{-1}$) its value at that grid point averages only $(1 - e^{-k^s \delta t}) / (k^s \delta t) \approx 86\%$ of δI^s . Carrying continuous spike times is a larger change, orthogonal to this one, and it becomes the natural next target now that the subthreshold integration is exact.

6.5 Summary

TABLE 3: Summary of the change.

	Euler	Exact propagator
Membrane error (float64)	6.4×10^{-2}	1.1×10^{-7}
Membrane error (float16)	6.4×10^{-2}	3.6×10^{-3}
Spikes at the right ms (float64) ^b	7.1 % (27.9 %)	99.9 % (100.0 %)
Spikes at the right ms (float16) ^b	5.2 % (26.8 %)	61.9 % (84.3 %)
Membrane error (mixed) ^a	6.4×10^{-2}	3.4×10^{-5}
Spikes at the right ms (mixed) ^{a,b}	7.2 % (28.1 %)	95.3 % (99.7 %)
Additional state variables	—	none
Fused kernel time (fp16)	1.00×	1.09×

^a float16 only for I^{syn} and C^{syn} ; V_j , I_j^s and the propagator constants in float32, arithmetic in float32. Not available in the fused operator today, which takes one dtype for every tensor, so it is measured with the same per-block rounding emulation as Table 1; in the all-float16 case that emulation reproduces the production CUDA kernel to 2×10^{-7} .

^b In brackets, the fraction reproduced to within ± 1 ms.

The implementation lives in `glif_propagators.py` under `v1_model_utils`. The scheme is selected by the `integration_scheme` ar-

gument of `create_model`, set to "exact" or "euler", or by the matching command-line flag. The accompanying notebook reproduces every figure and every number in this report.

7 REFERENCES

- [1] S. Rotter and M. Diesmann. "Exact digital simulation of time-invariant linear systems with applications to neuronal modeling". In: *Biological Cybernetics* 81 (1999), pp. 381–402.
- [2] J. Galván Fraile. "Neocortical Dynamics and Computational Mechanisms: Integrating Sensory and Higher-Order Inputs". PhD thesis. Universitat de les Illes Balears, 2025.
- [3] Ş. Mihalaş and E. Niebur. "A generalized linear integrate-and-fire neural model produces diverse spiking behaviors". In: *Neural Computation* 21 (2009), pp. 704–718.
- [4] C. Teeter et al. "Generalized leaky integrate-and-fire models classify multiple neuron types". In: *Nature Communications* 9 (2018), pp. 1–15. DOI: [10.1038/s41467-017-02717-4](https://doi.org/10.1038/s41467-017-02717-4).
- [5] D. Sterratt et al. *Principles of Computational Modelling in Neuroscience*. 2nd ed. Cambridge University Press, 2023.
- [6] Y. N. Billeh et al. "Systematic integration of structural and functional data into multi-scale models of mouse primary visual cortex". In: *Neuron* 106 (2020), pp. 388–403.
- [7] M.-O. Gewaltig and M. Diesmann. "NEST (NEural Simulation Tool)". In: *Scholarpedia* 2.4 (2007), p. 1430.
- [8] N. J. Higham. "The scaling and squaring method for the matrix exponential revisited". In: *SIAM Journal on Matrix Analysis and Applications* 26.4 (2005), pp. 1179–1193.
- [9] L. F. Richardson. "The approximate arithmetical solution by finite differences of physical problems involving differential equations". In: *Philosophical Transactions of the Royal Society A* 210 (1911), pp. 307–357.
- [10] P. Micikevicius et al. *Mixed Precision Training*. Tech. rep. International Conference on Learning Representations, 2018.
- [11] G. Bellec et al. "A solution to the learning dilemma for recurrent networks of spiking neurons". In: *Nature Communications* 11 (2020), p. 3625.
- [12] A. Morrison, S. Straube, H. E. Plesser, and M. Diesmann. "Exact subthreshold integration with continuous spike times in discrete-time neural network simulations". In: *Neural Computation* 19.1 (2007), pp. 47–79.
- [13] A. Hanuschkin et al. "A general and efficient method for incorporating precise spike times in globally time-driven simulations". In: *Frontiers in Neuroinformatics* 4 (2010), p. 113.